

高性能C/C++系统性能优化：从理论到实践

——MySQL数据库性能优化演进之路

何登成

写在最前

➤ Who am I?

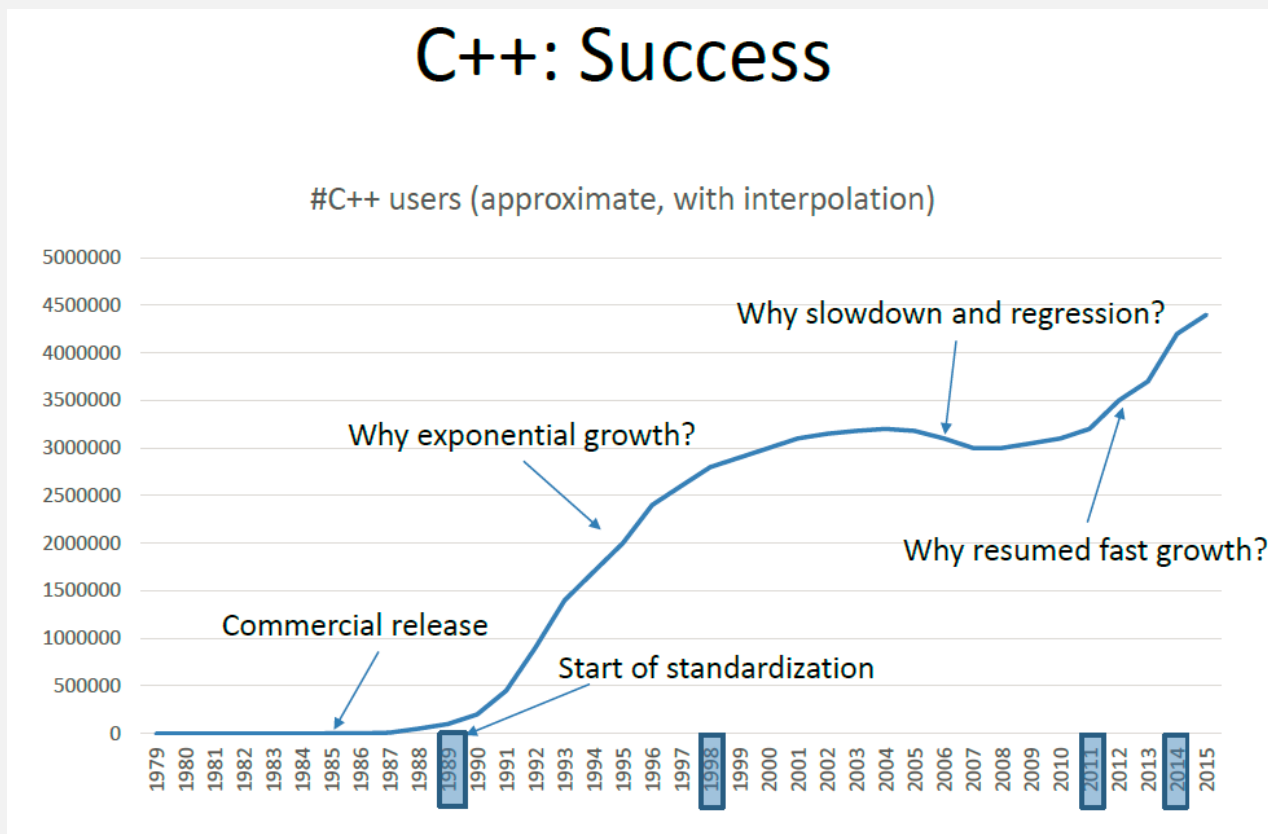
- 何登成，阿里巴巴数据库内核团队负责人，连续多年阿里/蚂蚁双11、新春红包数据库总负责人
- 微博：[何_登成](#)
- 博客：<http://hedengcheng.com>
- Github：<https://github.com/hedengcheng/tech>



C++发展曲线

➤ Why I'm here ?

- Not C++ Language



Bjarne Stroustrup. [The Evolution of C++ - Past, Present, and Future](#)

写在最前

➤ Why I' m here ?

➤ But **System Performance**

➤ Martin Thompson

➤ Hardware and software working together in harmony.

[Mechanical Sympathy](#)

➤ Chandler Carruth

➤ C++ DOESN' T GIVE YOU PERFORMANCE, IT GIVES YOU CONTROL OVER PERFORMANCE

[Efficiency with Algorithms, Performance with Data Structures](#)

分享大纲

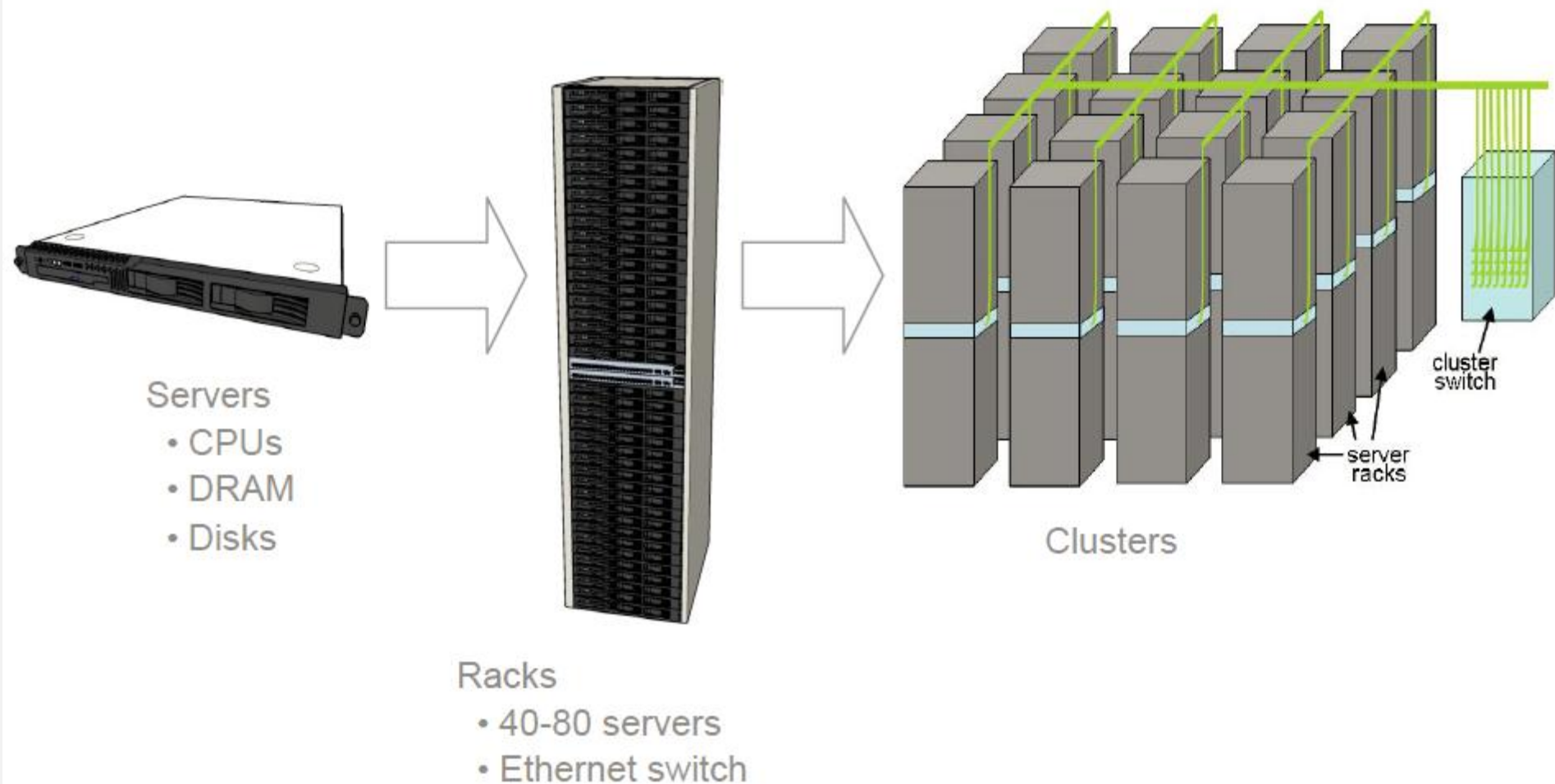
- 理论篇：工欲善其事必先利其器
 - 认识性能
 - 认识硬件
 - 数据结构&算法
 - 认识系统
 - 性能优化“三板斧”
- 实战篇：MySQL性能优化之路
 - MySQL前世今生
 - MySQL 5.1
 - MySQL 5.5 & 5.6
 - MySQL 5.7 & 8.0
 - AliSQL的性能优化

理论篇：认识性能

- 正确认识性能 ([Think Clearly about Performance](#))
 - *performance* usually means the time it takes for the system to execute some task.
- 性能的关键指标
 - Response Time (RT)：完成一个任务需要的时间
 - Throughput：指定时间内可以完成多少任务
 - Average vs Quantile (右图)
- 本文重点 (OLTP)
 - 在线事务处理。在一定的RT之内，追求Throughput的最大化

TABLE 1 Two Lists of Response Times (average for each is 1.000 second)		
	List A	List B
1	.924	.796
2	.928	.798
3	.954	.802
4	.957	.823
5	.961	.919
6	.965	.977
7	.972	1.076
8	.979	1.216
9	.987	1.273
10	1.373	1.320

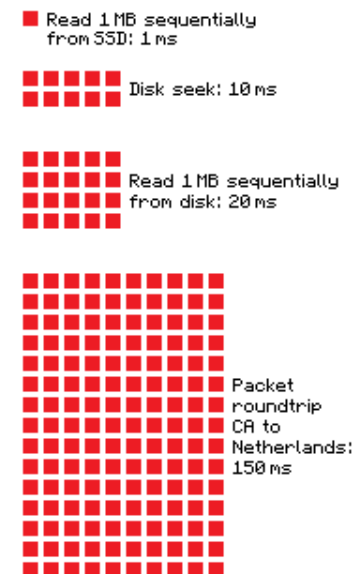
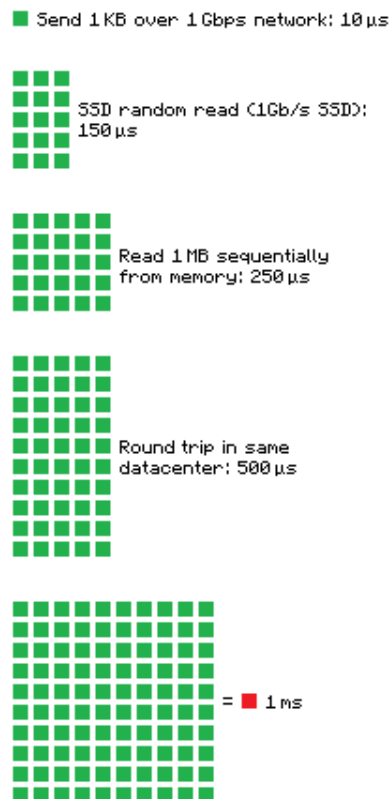
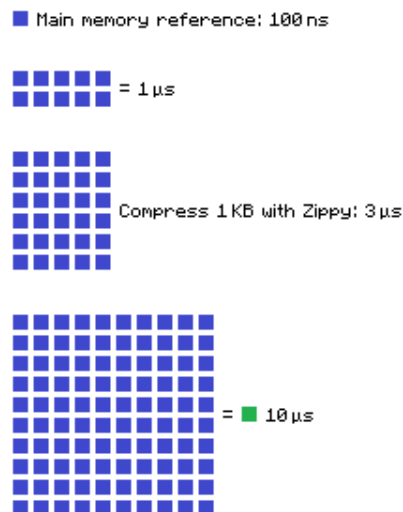
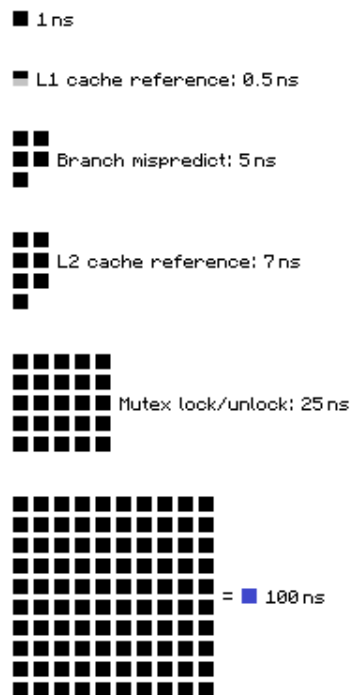
理论篇：认识硬件



➤ Jeff Dean. [Designs, Lessons and Advice from Building Large Distributed Systems](#)

理论篇：硬件性能指标

Latency Numbers Every Programmer Should Know



Source: <https://gist.github.com/2841832>

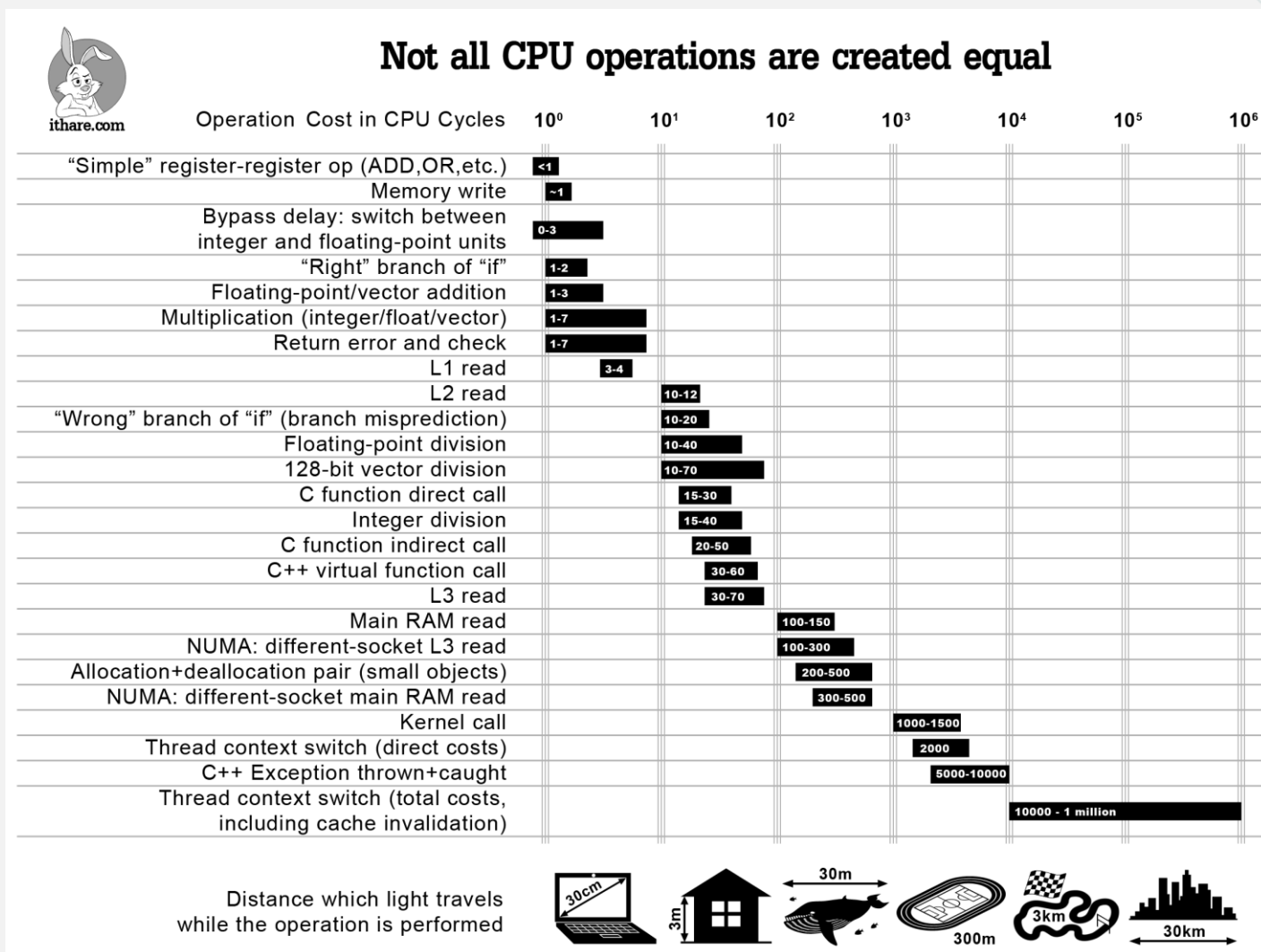
理论篇：硬件性能指标

Numbers Everyone Should Know

■ 1 ns	L1 cache reference	0.5 ns
■ L1 cache reference	Branch mispredict	5 ns
■ Branch mispredict	L2 cache reference	7 ns
■ L2 cache reference	Mutex lock/unlock	100 ns
■ Main memory reference	Main memory reference	100 ns
■ Compress 1K bytes with Zippy	Compress 1K bytes with Zippy	10,000 ns
■ Send 2K bytes over 1 Gbps network	Send 2K bytes over 1 Gbps network	20,000 ns
■ Read 1 MB sequentially from memory	Read 1 MB sequentially from memory	250,000 ns
■ Round trip within same datacenter	Round trip within same datacenter	500,000 ns
■ Disk seek	Disk seek	10,000,000 ns
■ Read 1 MB sequentially from network	Read 1 MB sequentially from network	10,000,000 ns
■ Read 1 MB sequentially from disk	Read 1 MB sequentially from disk	30,000,000 ns
■ Send packet CA->Netherlands->CA	Send packet CA->Netherlands->CA	150,000,000 ns

1/2841832

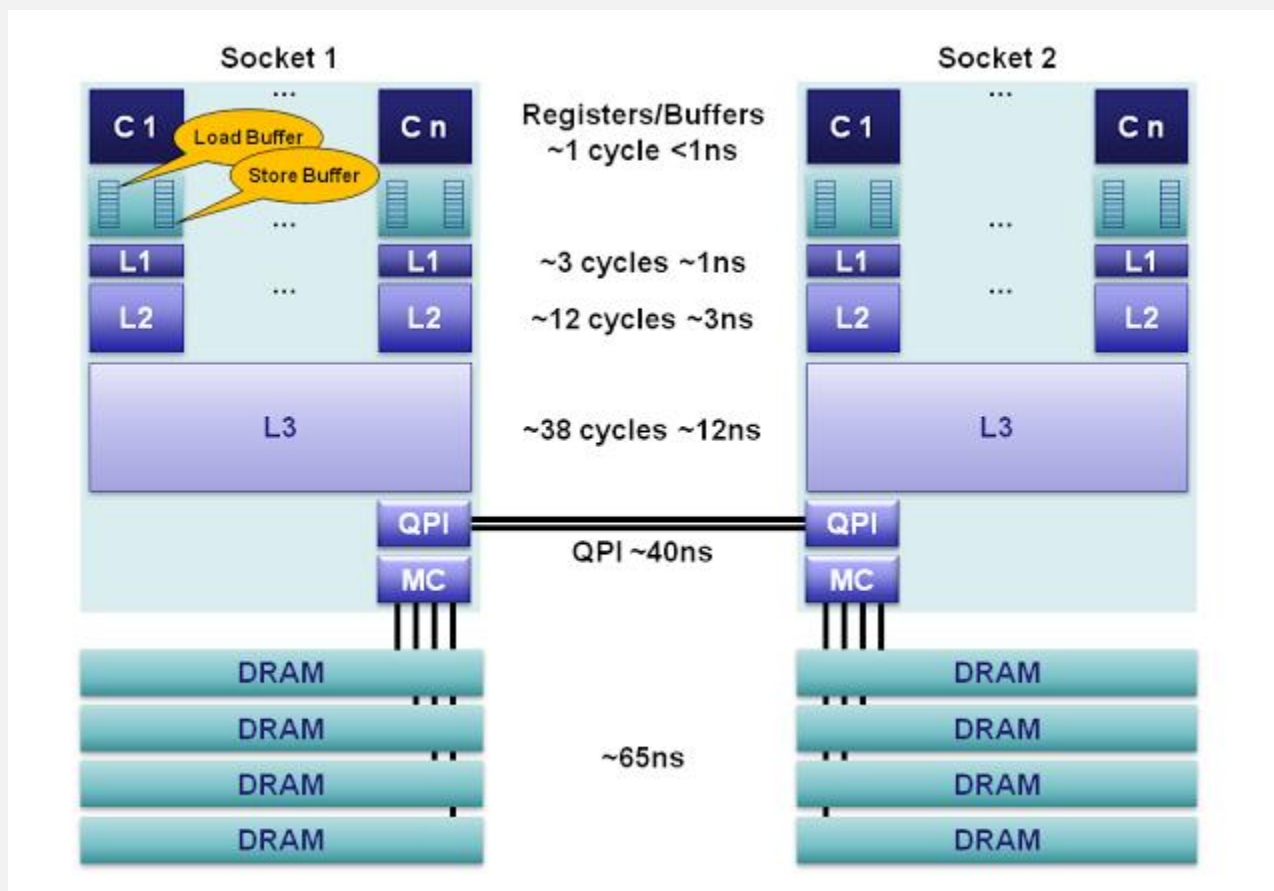
理论篇：硬件性能指标（另一个视角）



➤ “No Bugs” Hare. [Infographics: Operation Costs in CPU Clock Cycles](#)

理论篇：硬件性能指标

- Question: Memory Write: 1 Cycle, Memory Read: 100-150 Cycles, Why?



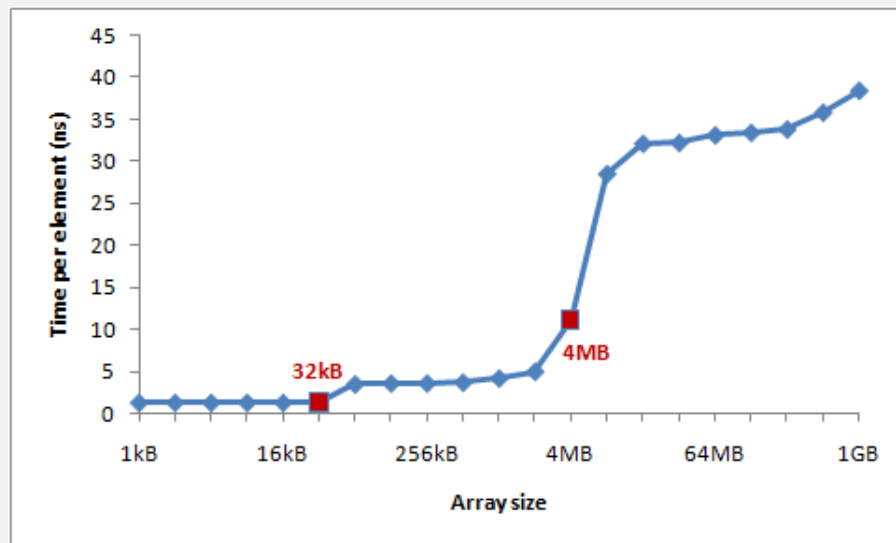
- 注：来自Martin Thompson [CPU Cache Flushing Fallacy](#) 一文；

理论篇：如何自己测试数据？

➤ 你应该亲自测试这些硬件性能指标

➤ Igor Ostrovsky——[Gallery of Processor Cache Effects](#)

- 测试内存Latency
- 测试Cache Lines大小
- 测试L1/L2/L3 Cache大小
- ...



➤ [ccBench](#) —— [Everything You Always Wanted to Know about Synchronization but Were Afraid to Ask](#) (SOSP' 13)

- 小工具，可以测试CPU Cache/Cache Coherence/Atomic Operations性能

理论篇：这些数据有什么用？

- **Question** : How long to quicksort 1 GB of 4 byte numbers?
 - From Jeff Dean 《[Software Engineering Advice from Building Large-Scale Distributed Systems](#)》
- **不写代码，预估执行时间？？？**
- **Comparisons** : 大量的branches misprediction
 - $\log_2(2^{28})$ passes over 2^{28} numbers = 2^{33} comparisons
 - $\sim 1/2$ will mispredict, so 2^{32} mispredicts * 5 ns/mispredict = 21 secs
- **Memory bandwidth** : 绝大部分sequential streaming
 - 2^{30} bytes * 28 passes = 28 GB. Memory BW is ~ 4 GB/s, so ~ 7 secs
- 预测完成1GB (4 byte number) 排序的总时间：约 30s

理论篇：事实果真如此吗？



理论篇：那实际排序1GB到底需要多少时间呢？

➤ 服务器

➤ 随机： **60s**

➤ 顺序： **17s**

```
Quicksort random 1GB(4 bytes int) on One CPU takes 60 sec.  
The smallest and largest 5 numbers are:
```

0:	2	2147483646
1:	10	2147483646
2:	20	2147483643
3:	23	2147483639
4:	38	2147483636

```
Quicksort sorted 1GB(4 bytes int) on One CPU takes 17 sec.  
The smallest and largest 5 numbers are:
```

0:	2	2147483646
1:	10	2147483646
2:	20	2147483643
3:	23	2147483639
4:	38	2147483636

理论篇：那实际排序1GB到底需要多少时间呢？

➤ 服务器

➤ 随机: **60s**

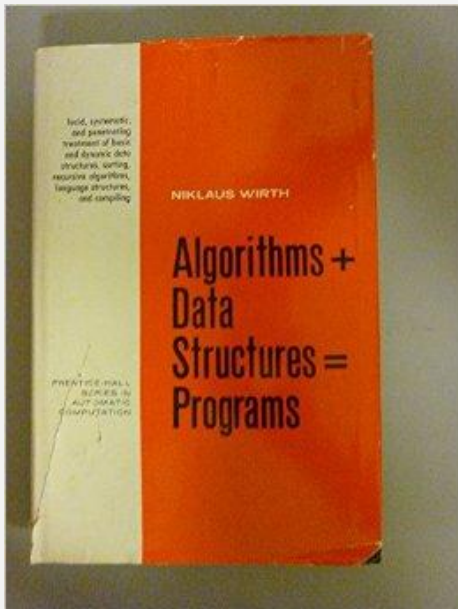
➤ 顺序: **17s**

```
Quicksort random
The smallest array
0:      2
1:     10
2:     20
3:     23
4:     38

Quicksort sorted
The smallest array
0:      2
1:     10
2:     20
3:     23
4:     38
```



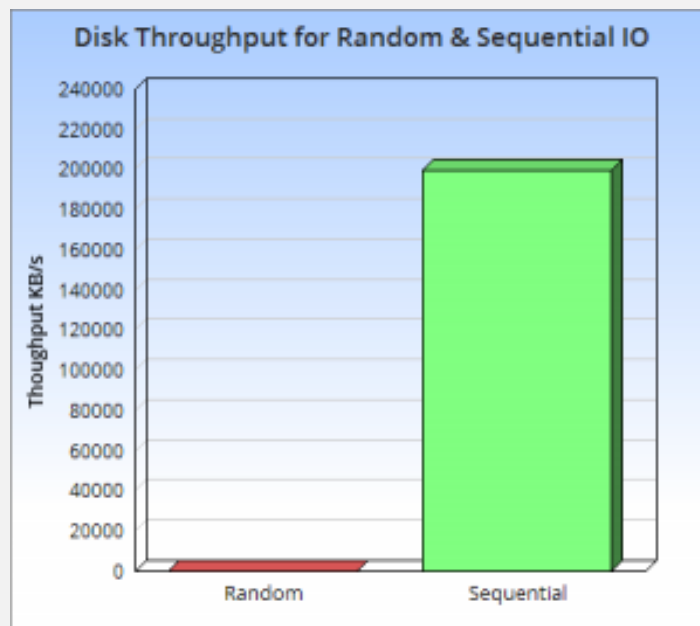
理论篇：数据结构 & 算法



Algorithms + Data Structures = Programs.

Niklaus Wirth. 1976

- 一个问题，如右图所示：
 - Hard Disk的随机读写能力远远弱于顺序读写能力。
 - 以数据库为例，对数据结构 & 算法有何启示？如何优化？



理论篇：数据结构 & 算法 (B+-Tree vs LSM-Tree)

A B+ Tree

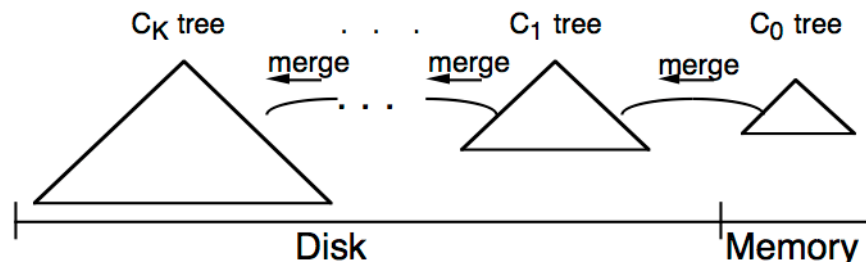
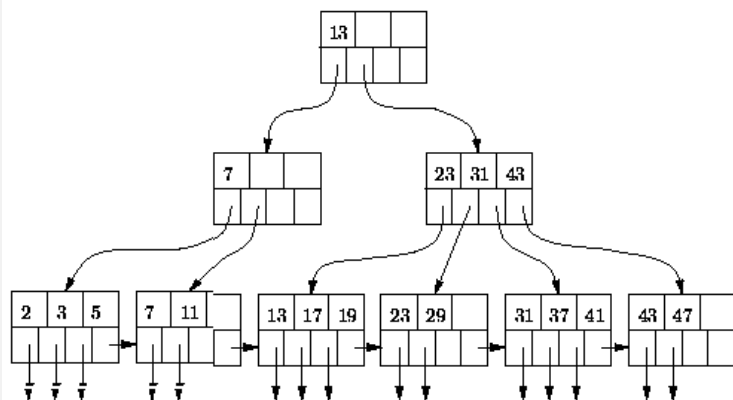


Figure 3.1. An LSM-tree of K+1 components

- One size fit all?
- ben stopford. [Log Structured Merge Trees](#)
- Harvard Research. [The RUM Conjecture](#)

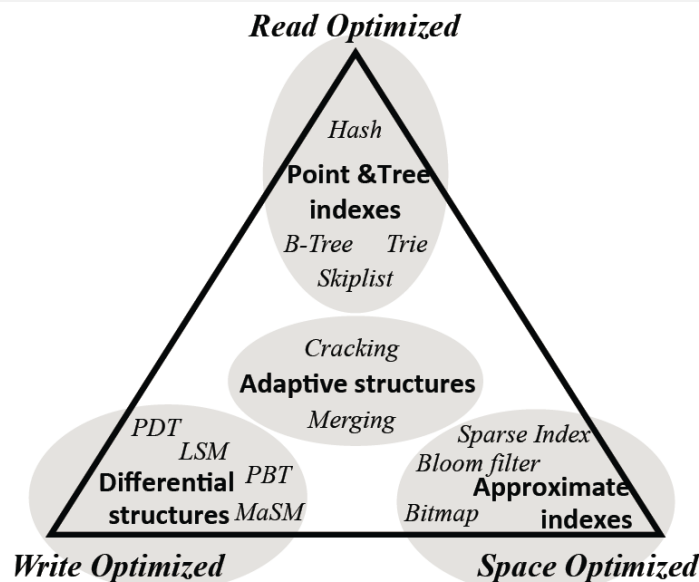
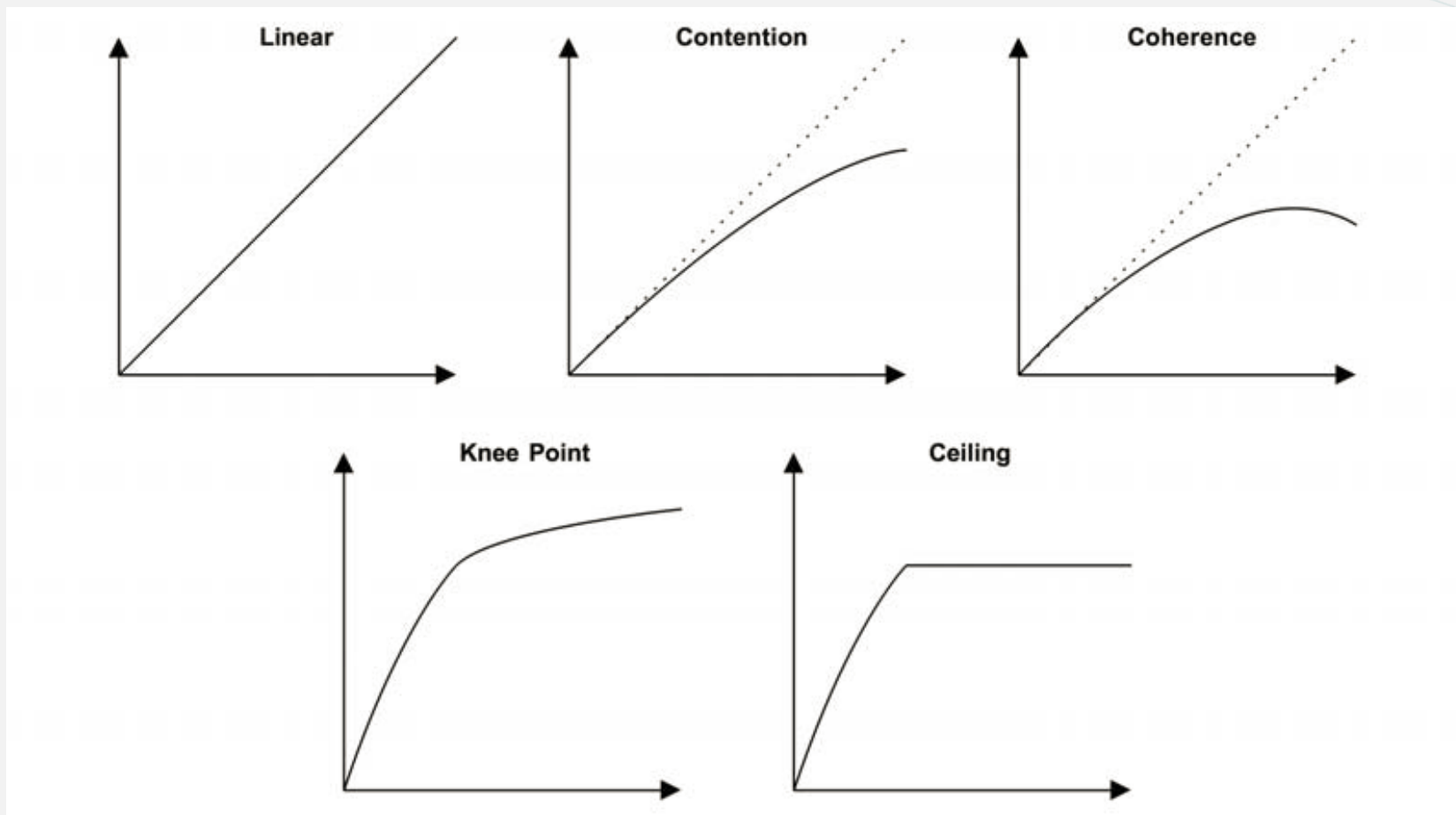


Figure 1: Popular data structures in the RUM space.

理论篇：认识系统

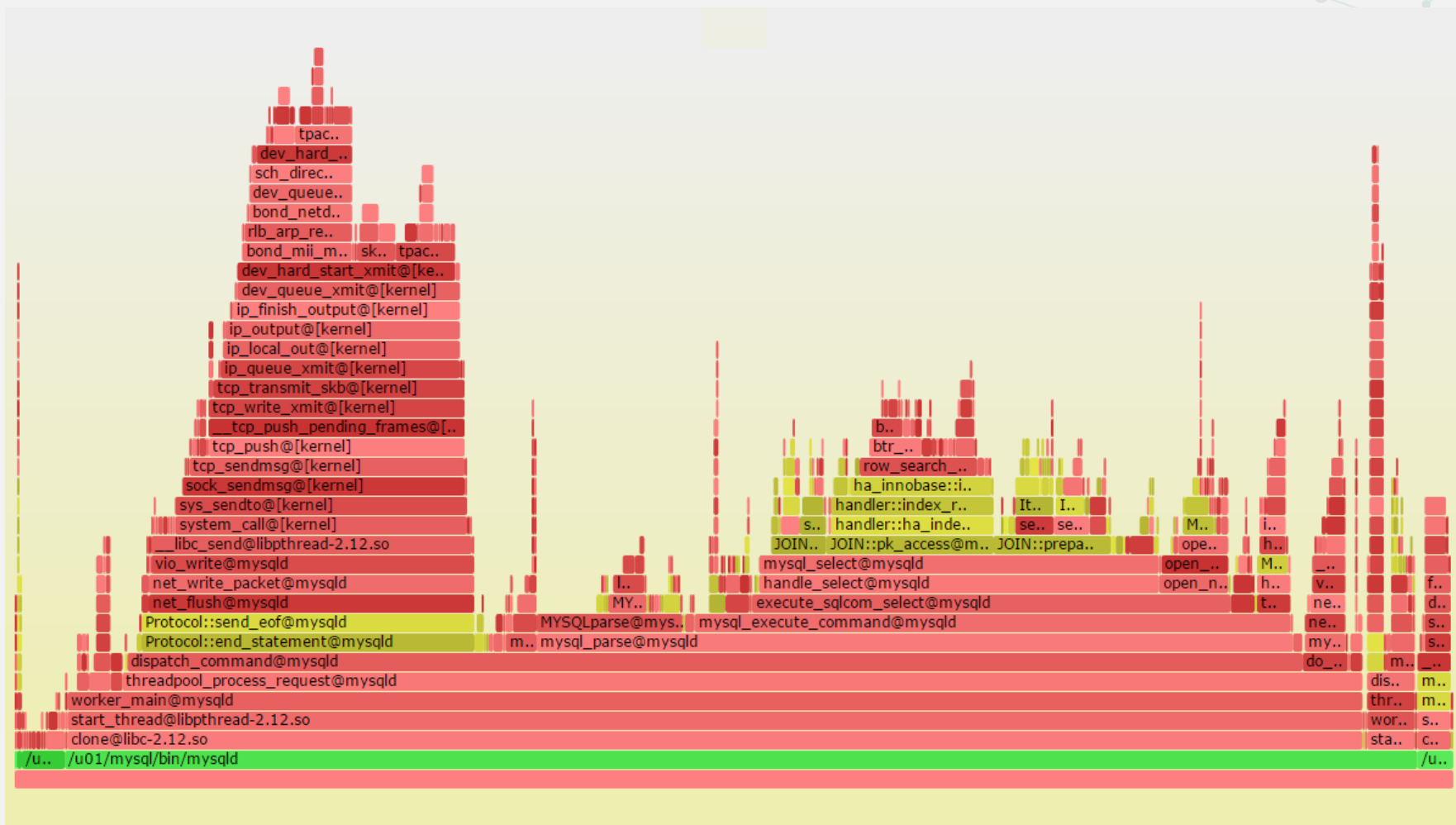
- Newly Designed System
 - 本文略过不表
- Existed System (尝试问自己几个问题)
 - 系统的设计容量?
 - 系统能够接受的Response Time?
 - 系统的并发瓶颈? 硬件瓶颈? 软件瓶颈?
 - 系统未来的压力增长?
 - 系统的容灾能力?
 -

理论篇：认识系统——宏观层面



➤ 来自Gregg Brendan 《Systems Performance: Enterprise and the Cloud》一书

理论篇：认识系统——微观层面



- Brendan Gregg. [Flame Graphs](#).
- Brendan Gregg. [Blazing Performance with Flame Graphs](#)

理论到实战的过度



➤ 上图

- 我们需要掌握的建造技术

➤ 右图

- 我们在实战中的落地策略



分享大纲

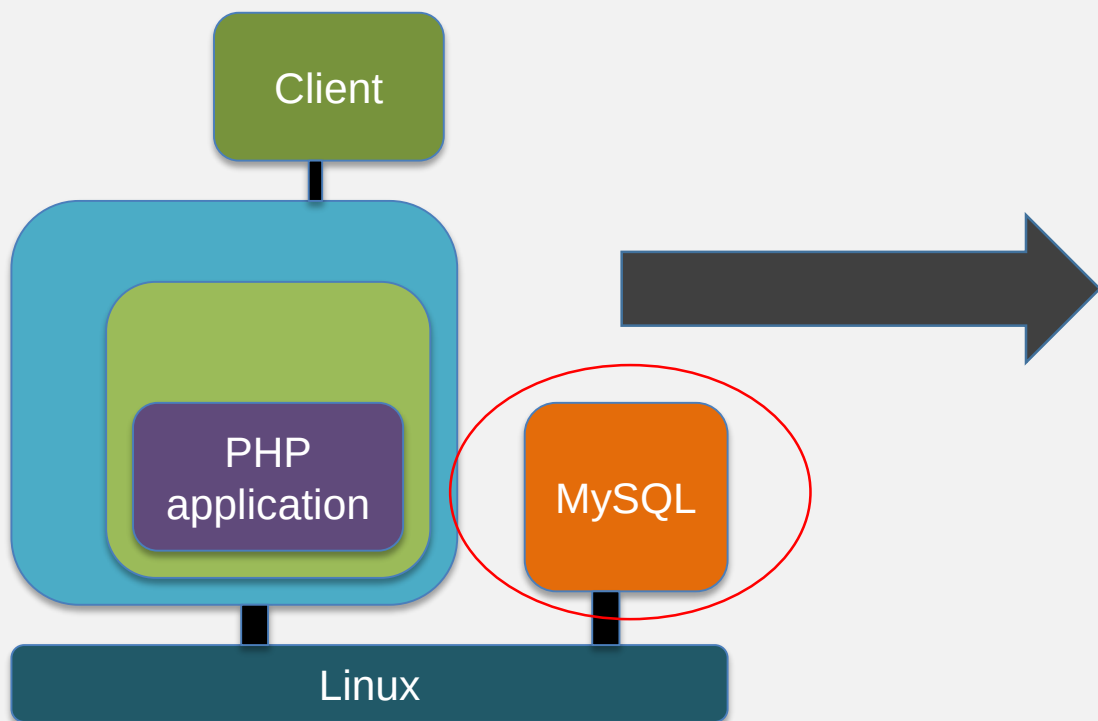
- 理论篇：工欲善其事必先利其器
 - 认识硬件
 - 数据结构&算法
 - 认识系统
 - 性能优化“三板斧”
- 实战篇：MySQL性能优化之路
 - MySQL前世今生
 - MySQL 5.1
 - MySQL 5.5 & 5.6
 - MySQL 5.7 & 8.0
 - AliSQL的性能优化

实战篇：MySQL的前身今生



- **MySQL: 21 years old — May 1995**
- 1995: MySQL 1.0 + MySQL AB founded
- **2001: InnoDB storage engine (InnoDB Oy)**
- March 2003: 4.0 GA + 4.1 & 5.0 as alpha
- **October 2005: Oracle acquires Innobase Oy (“InnoDB Friday”)**
- October 2005: 5.0 becomes GA*
- **November 2008: 5.1 GA**
- April 2009: Oracle acquires Sun Microsystems
- December 2010: MySQL 5.5 GA
- February 2013: MySQL 5.6 GA
- October 2015: MariaDB 10.1 GA, MySQL 5.7 GA
- 详见：Colin Charles. [The MySQL Server Ecosystem in 2016](#)

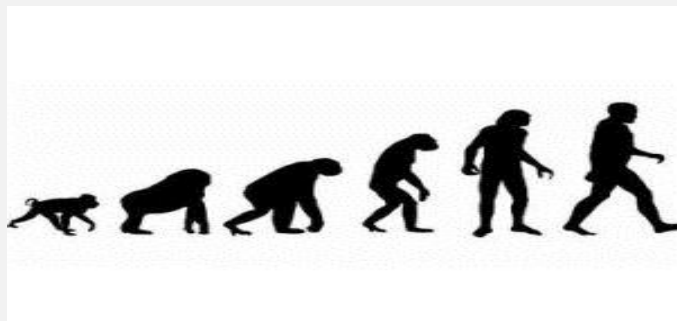
实战篇：MySQL发展史



最终成交额
912亿

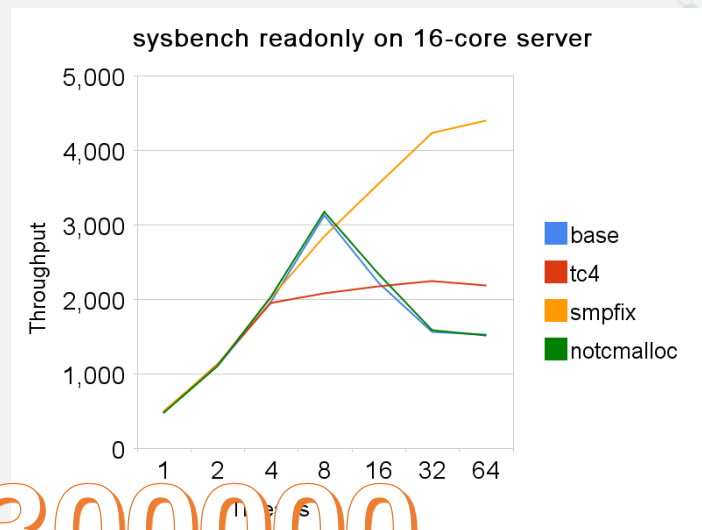
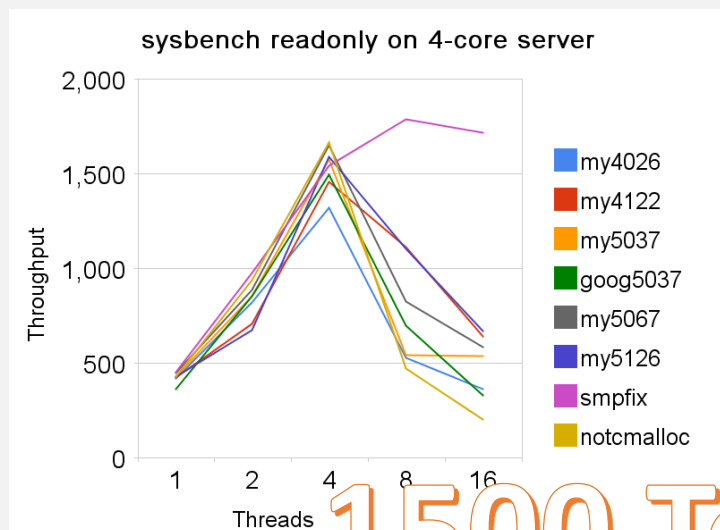
00:01	10亿	12:49	600亿
00:03	30亿	13:00	605亿 无线占比 71.22%
00:05	47亿	14:00	633亿 无线占比 70.88%
00:12	100亿	15:00	658亿 无线占比 70.52%
00:20	146亿	16:00	683亿 无线占比 70.09%
00:30	191亿	16:39	700亿 无线占比 69%
00:40	215亿	17:22	717亿 无线占比 69%
01:00	247亿	18:20	737亿 无线占比 69%
01:30	317亿	21:00	800亿 无线占比 68%
07:45	417亿	22:32	850亿 无线占比 68%
09:52	500亿	23:47	900亿 无线占比 68%
11:49	571亿	24:00	912亿

➤ LAMP：个人网站

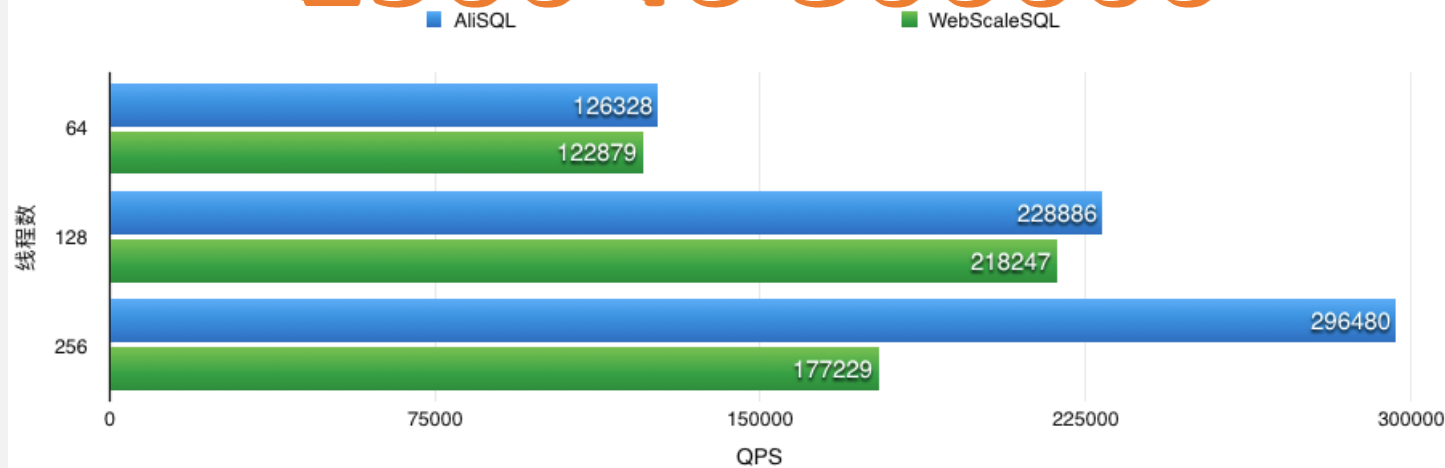


➤ 支持双11的高性能数据库

实战篇：MySQL性能演进



1500 To 300000



图片来源：<https://github.com/google/mysql-tools/wiki/SMP-Performance-Improvement>

实战篇：MySQL性能演进剖析

➤ Rome Wasn't Built in a Day

- MySQL的性能也不是一次优化到位的

➤ But They Were Laying Bricks Every Hour

- MySQL每一个版本，都在不断地做着性能优化

➤ MySQL各版本的性能优化，按技术主旋律可归纳为三个阶段

➤ 第一阶段：MySQL 5.5 & Before

- 主旋律： Coarse-grained to Fine-grained locking; Single Thread to Multi-Thread

➤ 第二阶段：MySQL 5.6 & 5.7

- 主旋律：深入系统实现细节的并发优化，软硬件结合

➤ 第三阶段：MySQL 8.0

- 主旋律：面向未来的架构重构

实战篇：MySQL 5.5 & Before

- 早期MySQL 特性
 - 简洁，不高效
 - 紧抓核心架构，放宽性能诉求
- Coarse-grained to Fine-grained locking
- Single Thread to Multi-Thread.
 - Mark Callaghan. [High Value Transaction Processing](#)

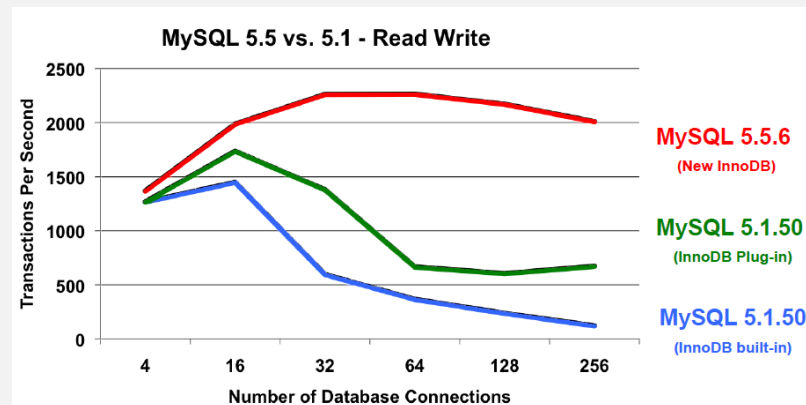
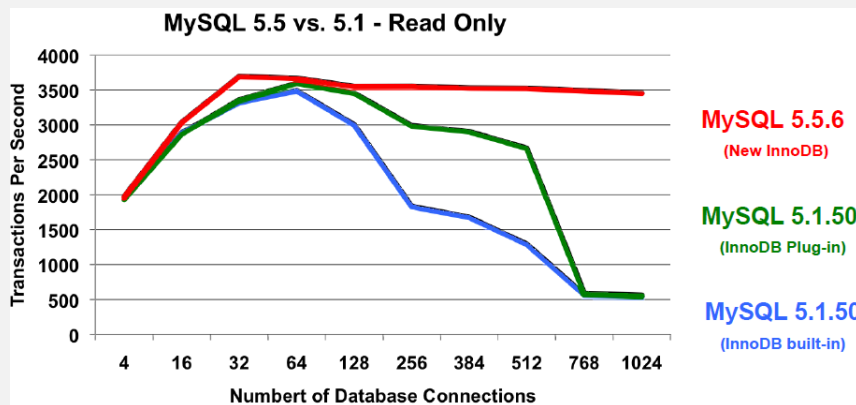


Stalls from mutexes

- | | |
|--------------------------------|-------------------------------|
| ▪ Extending InnoDB files | ▪ Buffer pool invalidate |
| ▪ Opening InnoDB tables | ▪ LOCK_open and kernel_mutex |
| ▪ Purge/undo lock conflicts | ▪ Excessive calls to fcntl |
| ▪ TRUNCATE table and LOCK_open | ▪ Deadlock detection overhead |
| ▪ DROP table and LOCK_open | ▪ innodb_th |

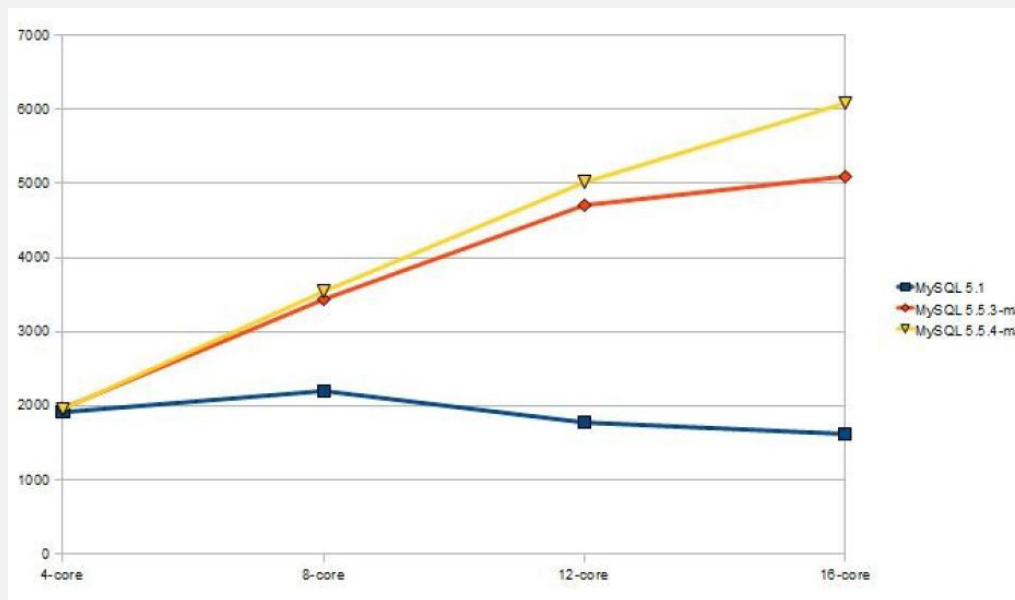
- Improved Purge Scheduling
- Improved default thread concurrency
- Control of background I/O threads

实战篇：MySQL 5.5 & Before



➤ 图片来自：[What's New in MySQL 5.5 : Performance and Scalability](#)

➤ 技术简单，成果斐然



实战篇：MySQL 5.6 & 5.7

➤ MySQL 5.6 & 5.7

- 经过第一阶段的优化，简单的优化手段已经逐渐收尾，进入第二阶段

➤ 第二阶段性能优化主旋律

- 深入系统实现细节的并发优化
- 软硬件结合

➤ 技能要求

- 了解硬件最新发展，获取硬件发展红利
- 熟练使用`perf`类工具，挖掘系统潜在瓶颈（强烈推荐[Brendan Gregg](#)，世界上玩`perf`玩得最好，分享最多的人！）



➤ 两个典型性能优化场景：

- 1. SSE指令集
- 2. G5 Discovery

实战篇：MySQL5.6 & 5.7（SSE 指令集）

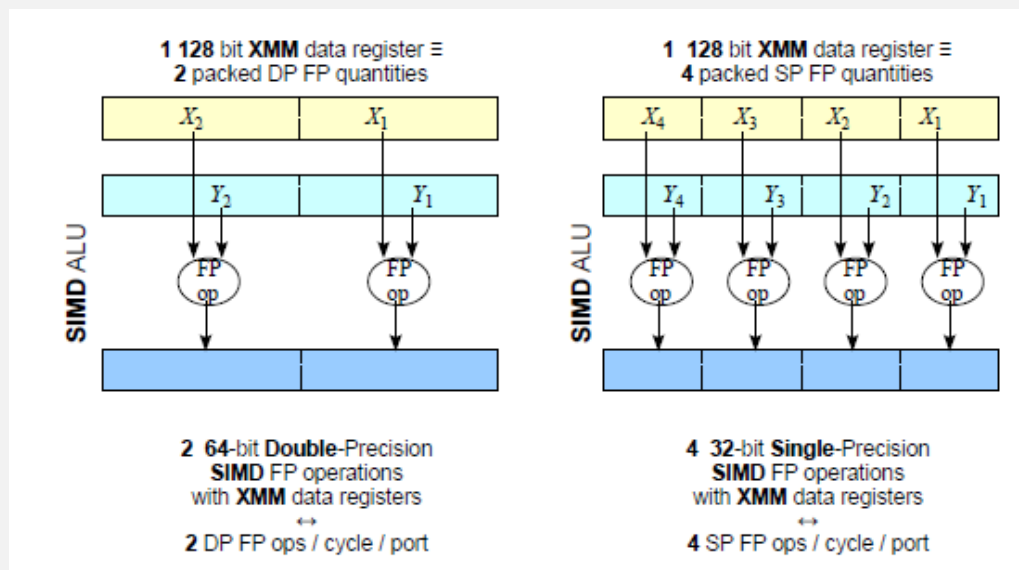
➤ 高级CPU指令

➤ SIMD SSE 4

➤ Single-Instruction-Multiple-Data

➤ 合适的应用场景

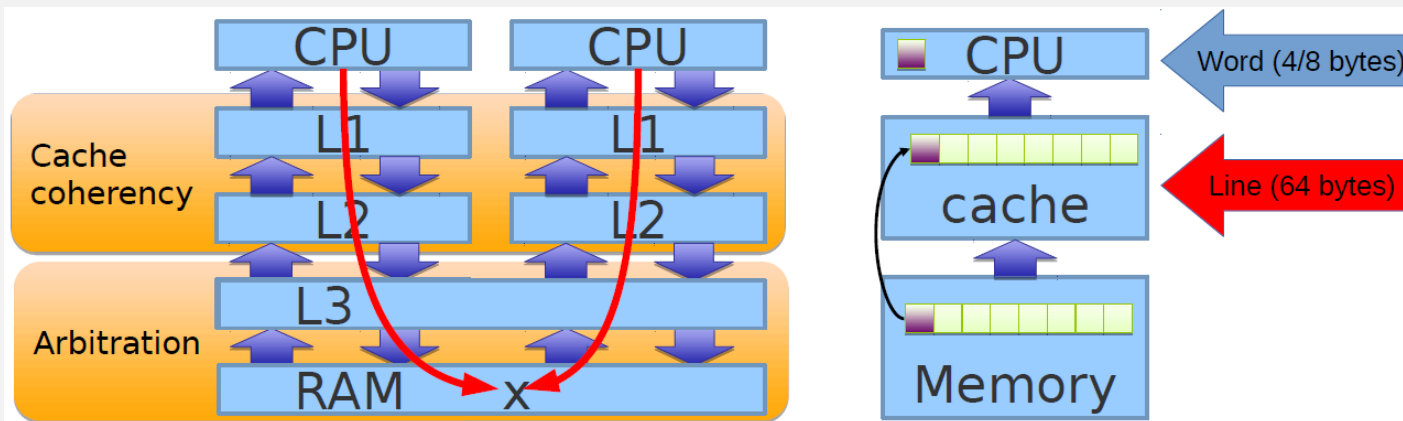
- InnoDB页面计算Checksum
- 硬件指令加速



```
/* opcodes taken from objdump of "crc32q (%%rdx), %%rcx"
for RHEL4 support (GCC 3 doesn't support this instruction) */
#define ut_crc32_sse42_quadword \
    asm(".byte 0xf2, 0x48, 0x0f, 0x38, 0xf1, 0x0a" \
        : "=c"(crc) : "c"(crc), "d"(buf)); \
    len -= 8, buf += 8
```


实战篇：MySQL5.6 & 5.7 (G5 Discovery)

➤ Cache Coherence & False Sharing



➤ G5 Discovery

➤ `srv_n_rows_read++`

➤ Per-Thread Counter

➤ 减少Cache Coherence

➤ Counter Padding

➤ 消除False Sharing

➤ `ib_counter_t::m_counter[(N + 1) * (CACHE_LINE_SIZE) / sizeof(Type)]`

➤ 成果：MySQL team increases scalability by >50% for Sysbench OLTP RO in MySQL 5.6 labs release april 2012...

实战篇：G5 Discovery，故事并未结束

➤ 测试验证

- 机器：16核以上服务器 程序：16线程，单线程累加 $1e8$ 次，总计 $16e8$ 次

➤ 1. 单counter，原子加

- 51.123s

➤ 2. 多counter，cache line pad，getcpu散列

- 2.340s

➤ 3. 多counter，cache line pad，getcpu散列，无原子加

- 1.432s

- MySQL Bugs. [Inefficient InnoDB row stats implementation](#)

- Mark Callaghan. [Make MyRocks 2X less slow](#)

1. 单counter，原子加

```
$time ./a.out 100
sum:1600000000

real    0m51.128s
user    10m49.877s
sys     0m0.058s
```

• 51s

2. 多counter，有cache line pad, getcpu散列，原子加

```
$time ./a.out 100
sum:1600000000

real    0m2.340s
user    0m31.597s
sys     0m0.003s
```

• 2.34s

3. 多counter，有cache line pad, getcpu散列，无原子加

```
$time ./a.out 100
sum:1600000000

real    0m1.432s
user    0m19.907s
sys     0m0.003s
```

• 1.43s

实战篇：G5 Discovery，最新战果

QPS, read-only, oltp-range=size=1000

1	2	4	8	16	24	32	40	48	64	80	96	128	concurrency/engine
1899	3830	7705	15750	31711	44379	50768	56297	60893	60724	60676	60338	60134	innodb
1971	3990	8087	16801	34110	48034	55710	61949	67497	66826	66368	65784	65493	innodb.fb
2486	5068	10222	21165	42235	58733	65402	70749	75154	74975	73579	73313	72951	innodb.5710
779	1545	3105	6202	12405	17377	19248	20702	21913	21940	21872	21877	21815	rocksdb.old
1198	2436	4901	9913	19996	28036	31607	34742	37246	37049	36617	36627	36738	rocksdb.norow
1708	3462	6997	14366	29479	41749	47278	51504	55817	55099	55215	54800	54755	rocksdb.cpu
1619	3307	6642	13447	27273	38049	42646	47105	50374	50006	49924	49817	49422	tokudb
1617	1626	1622	1619	1615	1613	1610	1608	1607	1609	1607	1607	1609	myisam

QPS, read-only, oltp-range=size=10000

1	2	4	8	16	24	32	40	48	64	80	96	128	concurrency/engine
243	485	966	1905	3736	5072	5811	6404	6501	6582	6554	6514	6404	innodb
250	500	991	1954	3839	5173	5958	6502	6857	6806	6745	6705	6637	innodb.fb
354	708	1406	2769	5326	6965	7606	8109	8274	8203	8146	8109	8054	innodb.5710
121	237	465	908	1742	2168	2324	2489	2598	2613	2603	2552	2592	rocksdb.old
174	341	674	1319	2570	3547	4035	4403	4602	4597	4585	4549	4466	rocksdb.norow
219	438	866	1706	3354	4586	5195	5609	5914	5911	5866	5811	5717	rocksdb.cpu
206	410	818	1614	3157	4300	4832	5237	5476	5401	5391	5345	5293	tokudb
187	187	187	187	187	187	187	186	187	186	189	190	189	myisam

- Mark Callaghan. [Make MyRocks 2X less slow](#)
- Mark Callaghan. [RocksDB Newest Performance Test](#)
- 何登成. [CPU Cache Coherence and Memory Consistency](#)

实战篇：MySQL 8.0

- 在MySQL每个版本演进的过程中，MySQL很多架构短板也逐渐暴露，从MySQL 5.7开始，MySQL就有意识的优化转到第三阶段
- 第三阶段主旋律
 - 面向未来的架构重构
- Why named 8.0 ? What' s new in 8.0 ?
 - **Major refactoring - C++11**, better support for ARM64/PPC
 - utf8mb4
 - SET PERSIST
 - Bye, bye, .FRM files - transactional data dictionary
 - **Optimiser: table level hints, invisible indexes, cost based optimiser**
 - Progress of row based replication in PERFORMANCE_SCHEMA?
 - Roles



没有性能优化，胜过大量性能优化！

MySQL生态体系

Percona Server

MariaDB

MySQL

AliSQL

WebScaleSQL

Others

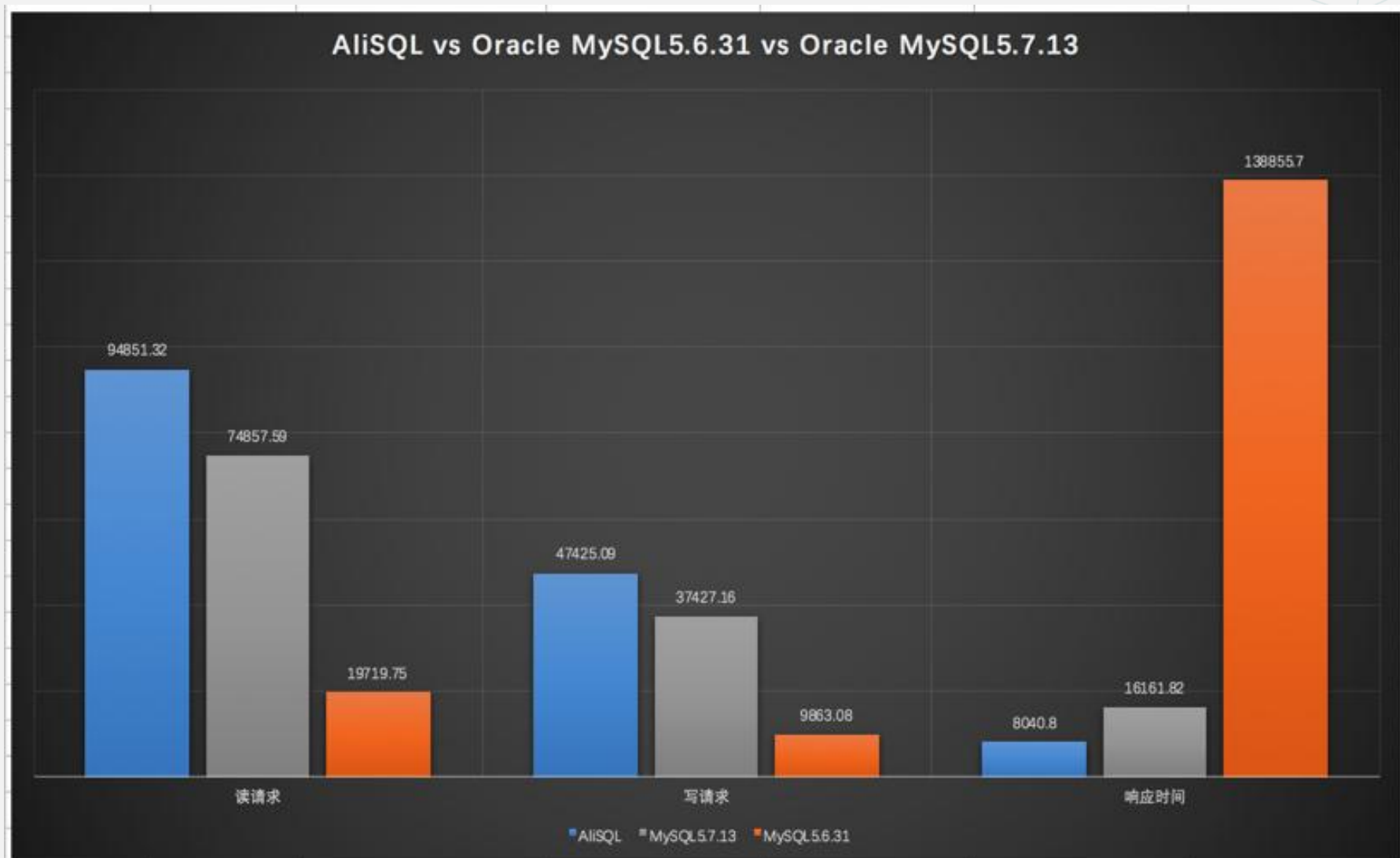
AliSQL：阿里巴巴MySQL分支

- AliSQL的定位
 - 高性能、高可用

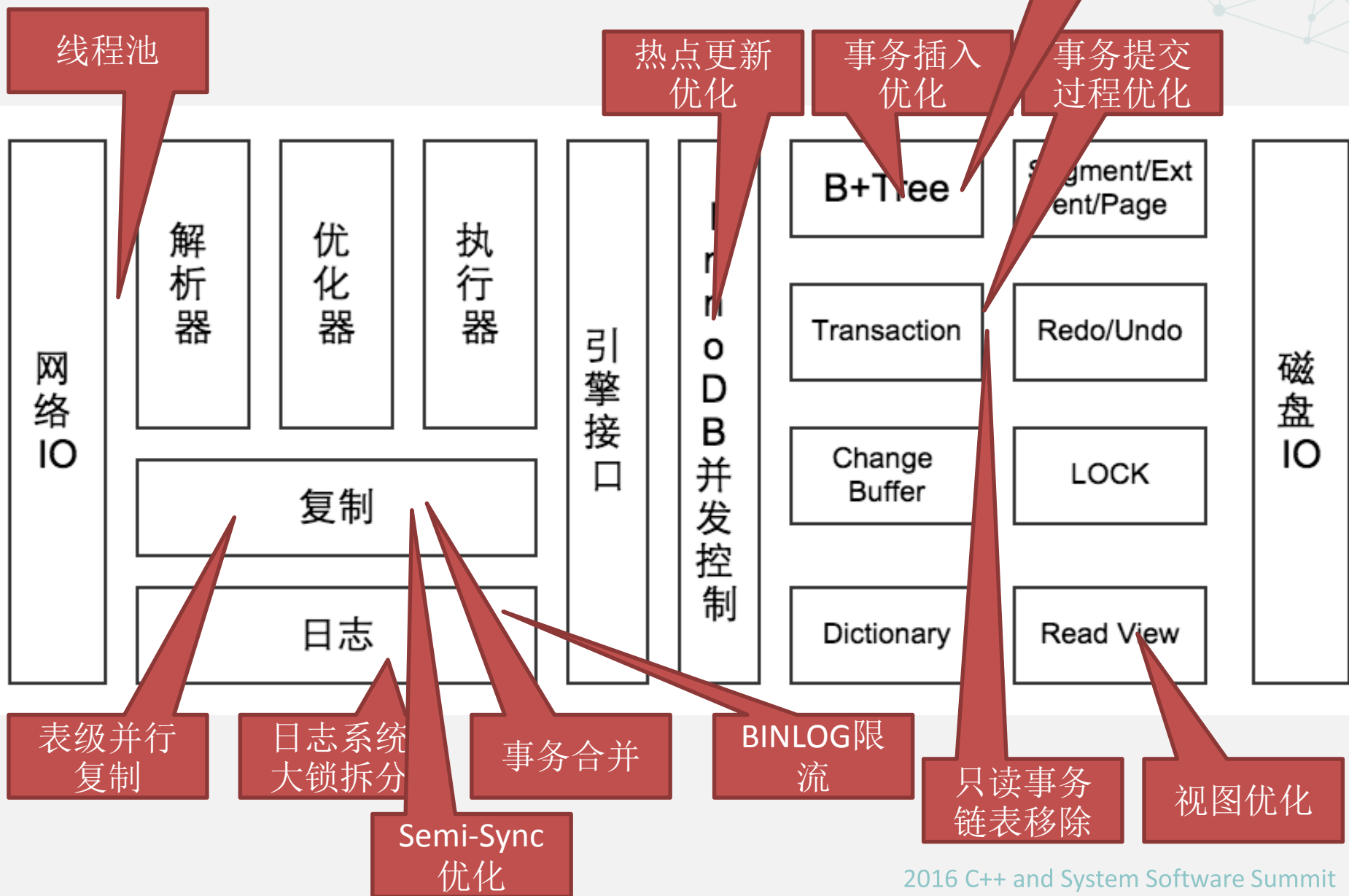
- AliSQL发展



AliSQL : 性能指标



AlisQL：性能优化



AliSQL：典型性能优化分析

➤ 场景优化

➤ Hot SKU

- 热点更新优化，电商场景下加快热点商品的销售速度，提升用户体验

➤ 通用优化

➤ Enhanced Thread Pool

- 增强线程池，应对大量应用服务器和海量并发请求

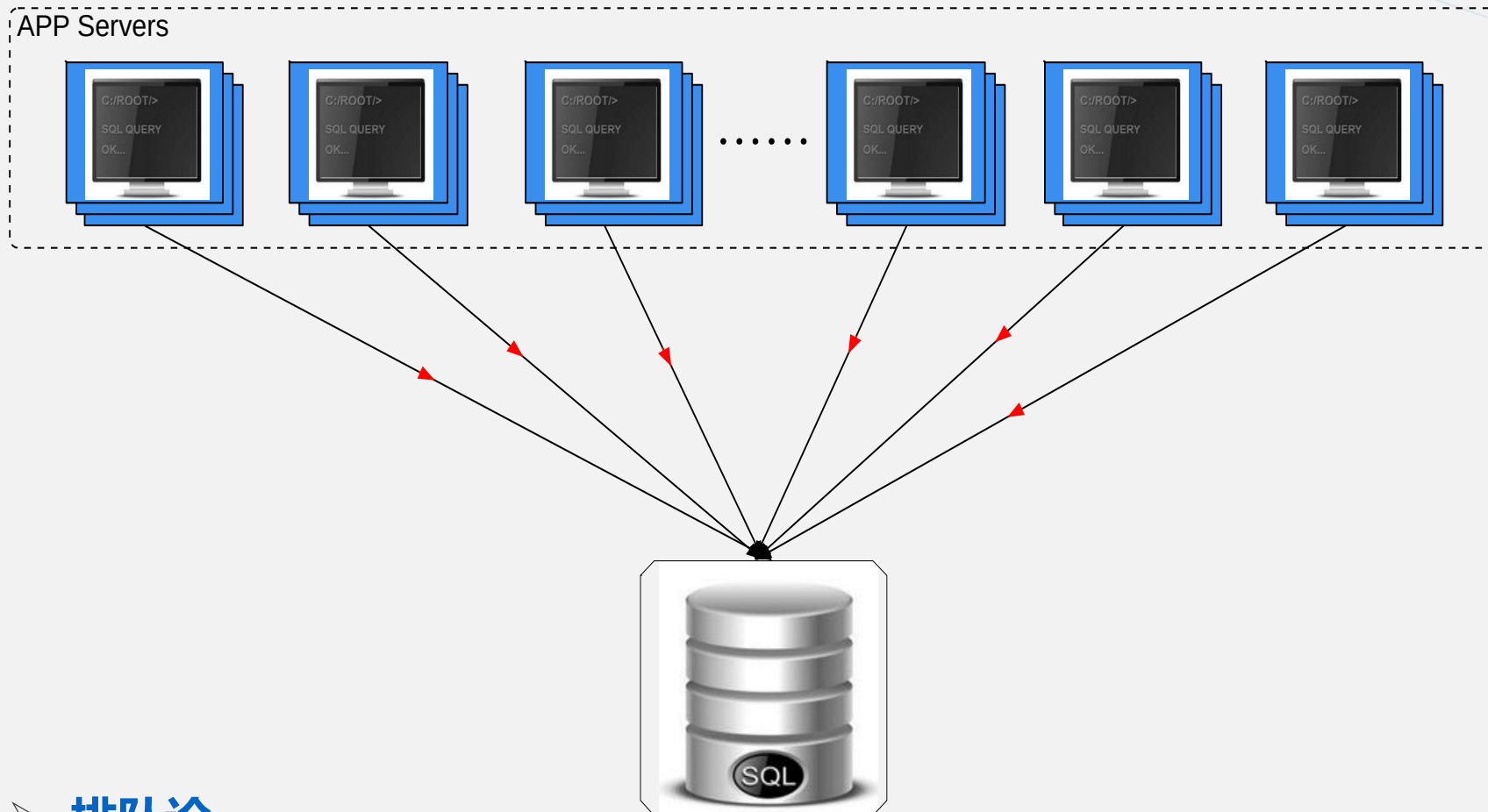
➤ PK Access

- 提升MySQL在处理简单查询下的极致性能

➤ Double Redo Buffer

- 提升MySQL在高压下的事务处理性能

AliSQL : Enhanced Thread Pool (1)

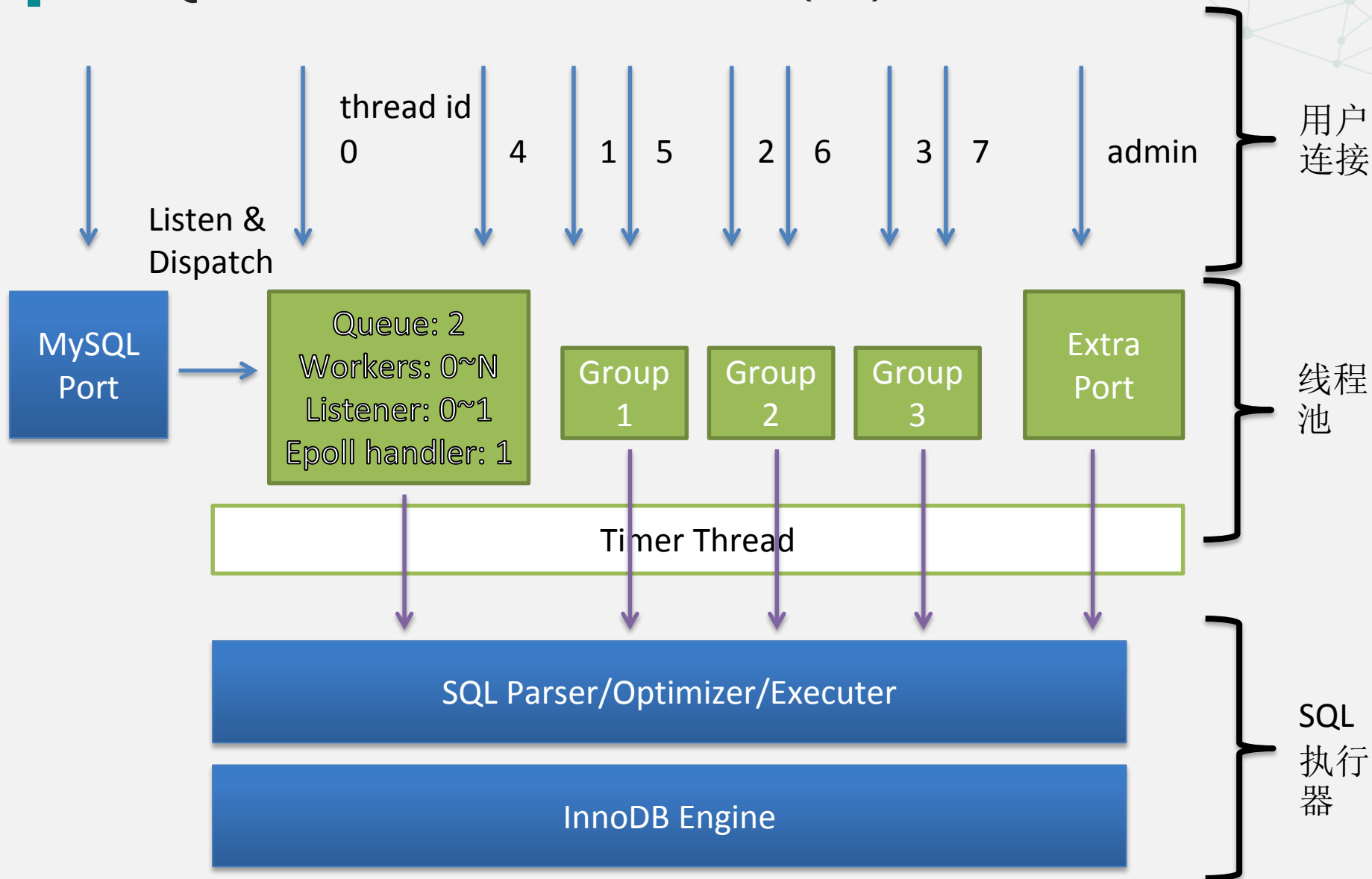


➤ 排队论

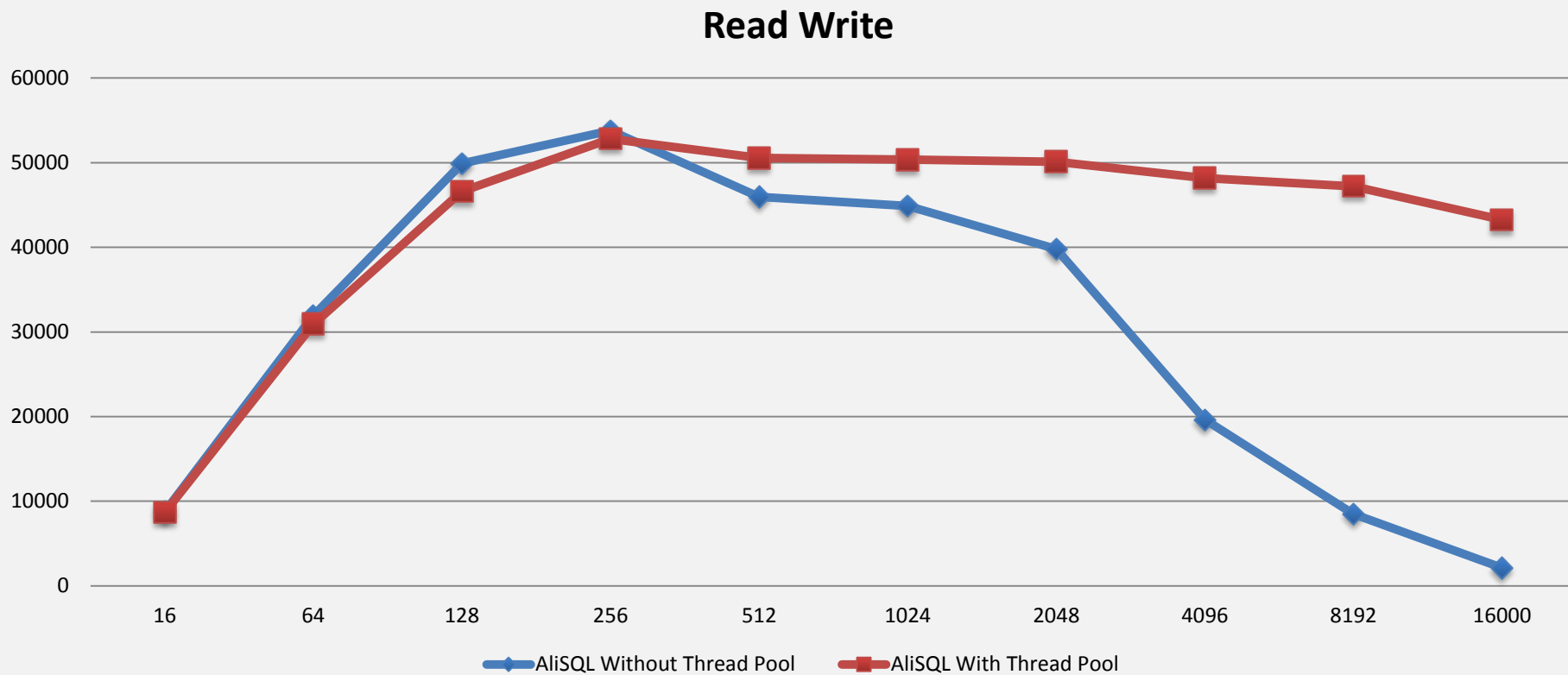
➤ [排队论及其应用浅析](#)

➤ [TCP Incast Congestion](#)

AliSQL : Enhanced Thread Pool (2)

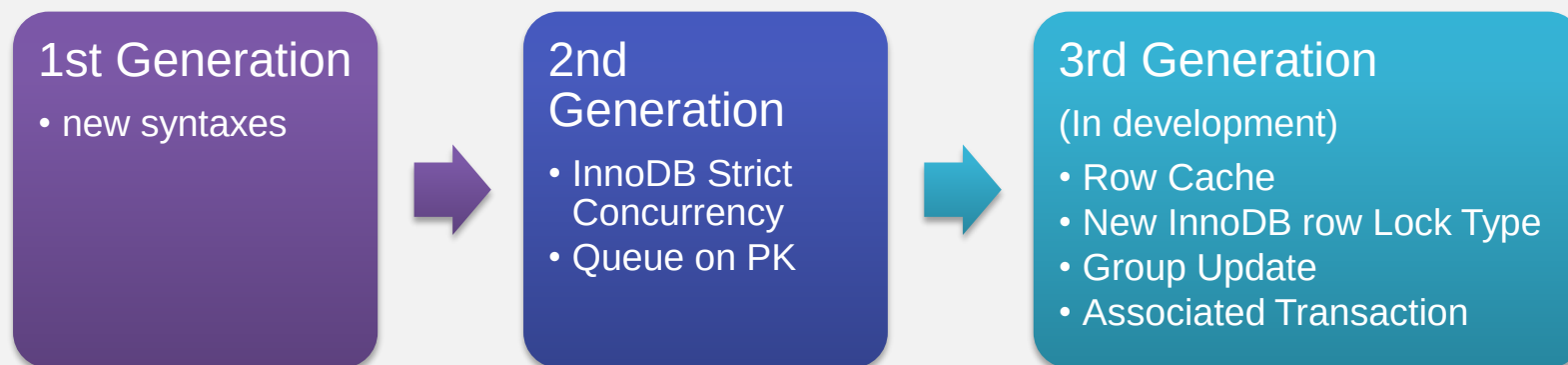


AliSQL : Enhanced Thread Pool (3)

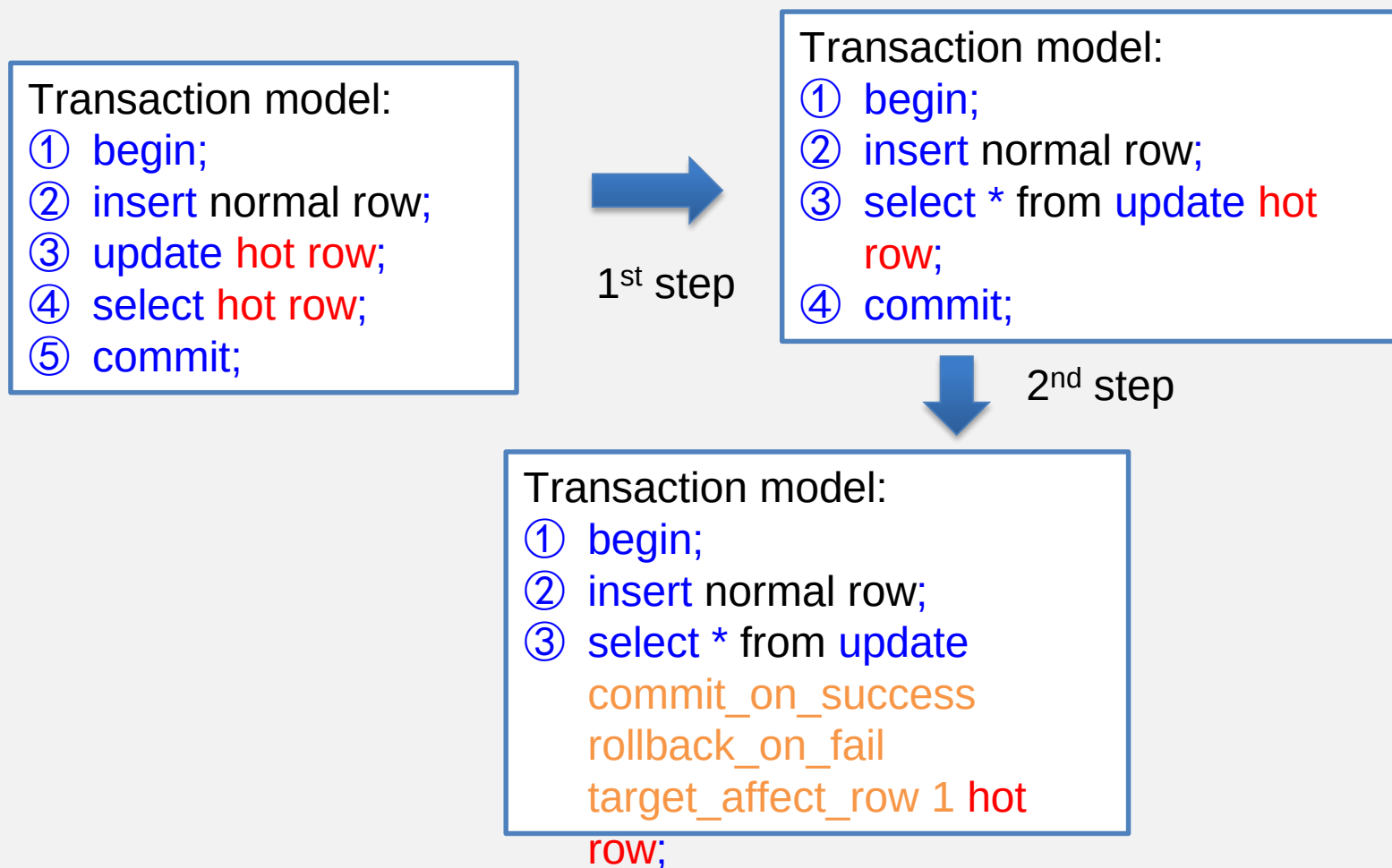


AlisQL : Hot SKU

➤ 历经三代演进



AliSQL : Hot SKU (第一代)



AliSQL : Hot SKU (第三代)



➤ 关键词

- 批处理
- 流水线
- 自动热点识别

AliSQL : Hot SKU (性能测试)



AliSQL : 更多的优化

- 关于AliSQL更多的性能优化、功能增强和稳定性提升，想要深入了解的朋友，可参考下面两篇分享
 - 2015 DTCC. [AliSQL 5.6及其应用](#)
 - 2016.4 Percona Live. [What's new in AliSQL - Alibaba's branch of MySQL](#)

写在最后



- [CppCon2016](#) , 非常多值得学习的分享（语言、产业相结合）。例如：
 - C++的发展史
 - Bjarne Stroustrup. The Evolution of C++ - Past, Present, and Future
 - 软硬件结合
 - Matt P. Dziubinski. Computer Architecture, C++, and High Performance
 - Timur Doumler. Want fast C++? Know your hardware!
 - 全局Counter，无锁编程
 - Fedor G Pikus. The Speed of Concurrency: is lock free faster?
 - 更多：[CppCon2016 Github](#)



谢谢大家！

Questions?